ناطق · NAATIQ — DOCUMENTATION

Technical Documentation

Architecture, data model, request flow, and every component of the Naatiq system

1. Architecture at a glance

Naatiq is a fully serverless system with no always-on server to maintain. It divides into three bands: the **channel** (WhatsApp via Twilio), the **brain** (two Supabase Edge Functions calling Groq and Claude), and the **surfaces** (dashboard, landing page, employer portal reading Postgres views).

```
WhatsApp user
     │  voice note / text
     ▼
Twilio WhatsApp  ──webhook──►  whatsapp-webhook (Edge Function)
                                    │ 200 OK returned immediately
                                    │ remaining work in EdgeRuntime.waitUntil
                                    ├─► Groq Whisper       (voice → Arabic text)
                                    ├─► Claude Sonnet      (interviewer turn +
extraction)
                                    ├─► Postgres           (users, profiles,
conversations)
                                    └─► invokes generate-cv when the profile is complete
                                              │
                                              ├─► Claude Sonnet   (bilingual CV content)
                                              ├─► Gotenberg       (HTML → Arabic-correct
PDF)
                                              ├─► Supabase Storage (cvs bucket)
                                              └─► Twilio          (PDF delivered on
WhatsApp)

Postgres ──security_invoker views──► dashboard · landing page · employer portal
```

The full diagram is `docs/naatiq-architecture.svg` (and `.png`).

2. Repository layout

Path	Contents
`supabase/migrations/`	

Path	Contents
	Four SQL migrations — schema, dashboard views, public CVs, placements
`supabase/functions/whatsapp-webhook/`	Inbound channel, transcription, interview loop
`supabase/functions/generate-cv/`	CV writing, PDF rendering, storage, delivery
`cv/`	Bilingual CV HTML template, local preview renderer, sample output
`dashboard/`	Arabic-first impact dashboard and brand assets
`web/`	Public landing page and employer portal
`infra/gotenberg/`	Dockerfile and deployment config for the PDF service
`docs/`	Architecture diagram, Markdown sources, generated PDFs, build script

3. Data model

`users`

`id`, `whatsapp_number` (unique), `display_name`, `state`, `created_at`.

The `state` column drives the whole conversation: `interviewing → profile_complete → cv_sent → practice_mode`. It is the single source of truth for what the agent should do next, which keeps the webhook stateless between invocations.

`profiles`

`user_id`, `full_name`, `first_name`, `role_trade`, `employers` (jsonb), `skills` (text[]), `tools` (text[]), `achievement`, `raw_extracted` (jsonb), `cv_pdf_path`.

`first_name` is a **generated column** — `split_part(coalesce(full_name,''),' ',1)` — so that public surfaces can show a human name without any code path ever having access to the full name. Privacy is enforced by the schema, not by remembering to strip a field in application code.

`conversations`

`user_id`, `direction` (`inbound` / `outbound`), `medium` (`voice` / `text`), `transcript`, `created_at`. Every turn is logged, which gives a full replayable audit trail of any interview.

`matches`

Reserved for job-family scoring. The table exists; matching logic is not implemented.

`placements`

`user_id`, `employer_name`, `job_title`, `placed_at`. Backs the employment counter so the number shown publicly is a real record rather than a derived guess.

Views

Three views, all created with `security_invoker = true`:

- `dashboard_stats` — aggregate counts (CVs generated, conversations, placements).
- `dashboard_profiles` — first name, trade, skills, `cv_pdf_path`. No full name, no phone.
- `dashboard_trade_distribution` — counts per trade for the chart.

4. `whatsapp-webhook`

Deployed with `verify_jwt: false` — Twilio cannot send a Supabase JWT.

Fast-acknowledge pattern

Twilio expects a webhook response within roughly 15 seconds. Transcription plus an LLM turn comfortably exceeds that. The function therefore parses the request, returns `200 OK` immediately, and performs all slow work inside `EdgeRuntime.waitUntil(...)`. Twilio never waits on Groq or Claude, and no message is ever lost to a webhook timeout.

Modules

File	Responsibility
`index.ts`	HTTP entry, fast ack, diagnostic endpoints
`env.ts`	Typed, validated environment access — fails loudly on a missing secret
`db.ts`	All Postgres reads and writes via the service-role client
`twilio.ts`	Outbound messages; defensively normalises the `whatsapp:` prefix
`groq.ts`	Downloads Twilio media and transcribes with `whisper-large-v3`
`anthropic.ts`	Claude Messages API client
`interviewer.ts`	The interview system prompt and extraction contract
`pipeline.ts`	Orchestrates a single inbound turn end to end
`tts.ts`	ElevenLabs Arabic voice replies (flag-gated, currently off)

Turn pipeline

1. Parse the Twilio form payload; find or create the user by `whatsapp_number`.

2. If media is attached, download it with Twilio basic auth and transcribe via Groq Whisper with `language: "ar"`.
3. Log the inbound turn to `conversations`.
4. Load conversation history and the current profile; call Claude with the interviewer prompt.
5. Claude returns both a natural reply **and** a structured JSON patch of newly learned facts. Merge the patch into `profiles`.
6. Send the reply over WhatsApp and log the outbound turn.
7. If the profile now has everything required, set `state = profile_complete` and invoke `generate-cv`.

Diagnostics

Supabase's log API exposes request lines but not `console.error` output, which made conventional debugging nearly useless. The fix was a secret-gated self-test endpoint on the function:

Query	Checks
`?selftest=<secret>`	Reports every env var's presence and the resolved config
`&claude=1`	Round-trips a message through Claude
`&sendto=<number>`	Sends a live WhatsApp test message
`&ttstest=1`	Exercises the ElevenLabs path
`&cvtest=<user_id>`	Triggers CV generation for an existing user

This endpoint independently identified four separate production bugs in minutes each. See the testing document for details.

5. `generate-cv`

Deployed with `verify_jwt: true` — it is only ever invoked server-to-server.

1. Load the profile.
2. `cvwriter.ts` calls Claude with the CV-writer prompt, which returns strict JSON containing both Arabic and English content: headline, summary, experience entries, skills, achievement.
3. `cv_template.ts` injects that JSON into the bilingual HTML template.
4. `render.ts` POSTs the HTML to Gotenberg's `/forms/chromium/convert/html` endpoint.
5. The resulting PDF is uploaded to the `cvs` Storage bucket and the path written to `profiles.cv_pdf_path`.
6. Twilio sends the PDF back to the user as a WhatsApp media message; `state` becomes `cv_sent`.

Passing `{ "user_id": "...", "debug": true }` runs the whole chain synchronously and returns a `steps[]` array showing exactly where it succeeded or failed — the same diagnostic philosophy as the webhook.

6. PDF rendering — why Gotenberg

Arabic PDF generation is the hardest technical requirement in the product. Arabic is right-to-left, its letters change shape by position, and most PDF libraries either reverse the string or render disconnected letterforms. A CV with broken Arabic is worse than no CV.

Gotenberg wraps headless Chromium, so it inherits the same text shaping and bidirectional layout engine as the browser — the most battle-tested Arabic rendering available. It is self-hosted on Railway from `infra/gotenberg/Dockerfile`, which binds Gotenberg to Railway's injected `$PORT`.

Alternatives were considered and rejected: **WeasyPrint** does not shape Arabic reliably; **pdfmake / jsPDF** need manual RTL handling; hosted HTML-to-PDF APIs add cost and a third-party dependency for the single most important artefact in the product.

`render.ts` accepts the Gotenberg URL with or without a scheme and normalises it, after a production failure caused by a bare hostname.

7. The CV template

`cv/template.html` is a single A4 page with a two-column bilingual body — Arabic (RTL) and English (LTR) side by side, so one document serves the worker and any employer.

Brand: navy `#001737` header with a 3px emerald `#09A385` bottom border, blue `#0E52E5` section headings and skill chips, and the Naatiq mark in the footer. Arabic uses Cairo/Amiri, English uses Lato, all embedded so rendering never depends on host fonts.

`cv/preview/render_preview.py` renders the template locally with sample data, which allowed the design to be iterated without spending WhatsApp round-trips.

8. Front-end surfaces

All three are single self-contained HTML files with no build step, reading Supabase views through the REST API with the publishable key.

- **`dashboard/index.html`** — Arabic-first RTL with an English toggle. Live counts, a Chart.js trade-distribution chart, and per-candidate CV links. Ships with an embedded data snapshot that renders instantly, then overlays live data — so a network failure degrades to slightly stale numbers rather than an error message on stage.
- **`web/index.html`** — public landing page: hero, live counters, how-it-works, who it is for, and a WhatsApp CTA.

- `web/employers.html` — browse and filter candidates by trade, open a CV, and request contact through Naatiq. Worker phone numbers are never rendered or fetched.

9. External services

Service	Role	Why
Twilio WhatsApp	Channel	Only practical route to WhatsApp for a hackathon; sandbox needs no Business verification
Groq Whisper `whisper-large-v3`	Arabic STT	Strong dialect handling, very low latency, generous free tier
Claude Sonnet	Interviewer + CV writer	Arabic fluency, reliable structured JSON, strong instruction-following
Gotenberg	HTML → PDF	Chromium-grade Arabic shaping
Supabase	DB, Storage, Functions, Auth	One platform for everything; Deno functions deploy in seconds
ElevenLabs	Arabic TTS	Built and flag-gated; requires a paid plan for API voice access

10. Configuration

Set as Supabase Function secrets:

```
SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY
TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_WHATSAPP_FROM   # must be whatsapp:+1...
GROQ_API_KEY
ANTHROPIC_API_KEY
GOTENBERG_URL
ELEVENLABS_API_KEY, ELEVENLABS_VOICE_ID
WATHEFTI_VOICE_REPLIES        # false
WATHEFTI_SELFTEST_SECRET
```

11. Notable implementation constraints

- **Claude Sonnet rejects the `temperature` parameter.** Sending it returns a 400. Both Anthropic clients omit it deliberately, with a comment so it is not reinnocently added back.
- **`TWILIO_WHATSAPP_FROM` must carry the `whatsapp:` prefix**, or Twilio returns error 63007. The code normalises it defensively regardless.
- **Supabase function deploys need `import_map_path: "deno.json"` passed explicitly** when updating an existing function, or the deploy fails on a missing import map.

- **Views must be `security_invoker = true`.** Without it, Supabase's advisor flags `security_definer_view`, because the view would bypass the caller's RLS.